

EH的第二个板子，作为补充

EH的第二个板子，作为补充

A 基础算法

A43 单调栈

B 搜索

B30 01BFS

C 数据结构

C000 树状数组2

D 图论

D3 SPFA算法

D18 Tarjan eDCC缩点

D19 Tarjan vDCC缩点

D21 Dinic最大流

D23 最小费用最大流EK

D27 二分图最大匹配

D99 Kruskal重构树

E 动态规划

E43 单调队列优化dp

E51 斜率优化DP

F 字符串

F5 马拉车算法-最长回文子串

G 数学

G12 莫比乌斯函数

G24 高斯约旦消元法

G26 求组合数-线性逆推

G51 三角剖分

G57 自适应辛普森积分

G60 有向图游戏-SG函数

G74 拉格朗日插值法

A 基础算法

A43 单调栈

```
n = int(input())
a = list(map(int, input().split()))
f = [0] * n
q = []
for i in range(n - 1, -1, -1):
    while q and a[q[-1]] < a[i]:
        q.pop()
    if q:
        f[i] = q[-1]
    q.append(i)
print(*f)
```

算法过于简单，所以没有C++对应的代码。

B 搜索

B30 01BFS

```
#include <bits/stdc++.h>
using namespace std;
#define endl "\n"
#define int long long
#define range(i, a, b) for (int i = (a); i < (b); ++i)
#define def(name, ...) auto name = [&](__VA_ARGS__)
vector<vector<pair<int, int>>> e;
// vector<int> a(n, 0);
// vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
void solve() {
    int n, m, k; cin >> n >> m >> k;
    e.assign(n * m, vector<pair<int, int>>());
    range(i, 0, n){
        string s; cin >> s;
        range(j, 0, m){
            int idx = i * m + j;
            if (i > 0){
                if (s[j] == 'U')e[idx].push_back({idx - m, 0});
                else e[idx].push_back({idx - m, 1});
            }
            if (j > 0){
                if (s[j] == 'L')e[idx].push_back({idx - 1, 0});
                else e[idx].push_back({idx - 1, 1});
            }
            if (i < n - 1){
                if (s[j] == 'D')e[idx].push_back({idx + m, 0});
                else e[idx].push_back({idx + m, 1});
            }
            if (j < m - 1){
                if (s[j] == 'R')e[idx].push_back({idx + 1, 0});
                else e[idx].push_back({idx + 1, 1});
            }
        }
    }
    deque<pair<int, int>> q;
    vector<int> vis(n * m, -1);
    q.push_back({0, 0});
    while (q.size()){
        auto [u, dis] = q.front();
        q.pop_front();
        if (vis[u] != -1) continue;
        vis[u] = dis;
        for (auto [v, le]: e[u]){
            if (vis[v] == -1){
                if (le == 0){
                    q.push_front({v, dis});
                }
                else {
                    q.push_back({v, dis + 1});
                }
            }
        }
    }
    if (vis[n*m-1] > k){
        cout << "NO" << endl;
```

```

    }
    else {
        cout << "YES" << endl;
    }
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int t;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}

```

C 数据结构

C000 树状数组2

```

import sys
input = sys.stdin.readline

class BIT:
    def __init__(self, n):
        self.n = n
        self.s = [0] * (n + 1)

    def change(self, x, w):
        while x <= self.n:
            self.s[x] += w
            x += x & -x

    def query(self, x):
        res = 0
        while x > 0:
            res += self.s[x]
            x -= x & -x
        return res

if __name__ == "__main__":
    n, m = map(int, input().split())
    T = BIT(n)

    a = list(map(int, input().split()))
    # 用差分数组初始化: d[1]=a1, d[i]=a[i]-a[i-1]
    prev = 0
    for i, val in enumerate(a, 1): # 下标从1开始
        d = val - prev
        T.change(i, d)
        prev = val

    for _ in range(m):
        op, *rest = map(int, input().split())

```

```

    if op == 1: # 区间修改: op=1 l r k
        l, r, k = rest
        T.change(l, k)
        if r + 1 <= n:
            T.change(r + 1, -k)
    else: # 点查询: op=2 x
        x = rest[0]
        print(T.query(x))

```

```

// 树状数组 区修+点查 O(nlogn)
#include<bits/stdc++.h>
using namespace std;

const int N=500010;
int n,m,a[N];

struct BIT{
    int s[N]; //差分的区间和
    void change(int x,int w){
        for(;x<=n;x+=x&-x) s[x]+=w;
    }
    int query(int x){
        int res=0;
        for(;x;x-=x&-x) res+=s[x];
        return res;
    }
}T;
int main(){
    ios::sync_with_stdio(0);
    cin>>n>>m; int op,x,y,k;
    for(int i=1;i<=n;i++) cin>>a[i];
    for(int i=1;i<=m;i++){
        cin>>op>>x;
        if(op==1){
            cin>>y>>k;
            T.change(x,k);
            T.change(y+1,-k); //差分
        }
        else cout<<T.query(x)+a[x]<<"\n";
    }
}

```

D 图论

D3 SPFA算法

```

#include <bits/stdc++.h>
using namespace std;
const int N = 2010;
int n, m;
int d[N], vis[N], cnt[N];
struct Edge {
    int v, w;
};
vector<Edge> g[N];
bool spfa() { // 判负环

```

```

memset(d, 0x3f, sizeof d); d[1] = 0;
memset(vis, 0, sizeof vis);
memset(cnt, 0, sizeof cnt);
queue<int> q;
q.push(1);
vis[1] = 1; // 在队中
while (!q.empty()) {
    int u = q.front(); q.pop();
    vis[u] = 0;
    for (auto &e : g[u]) {
        int v = e.v, w = e.w;
        if (d[v] > d[u] + w) { // 松弛
            d[v] = d[u] + w;
            cnt[v] = cnt[u] + 1; // 边数
            if (cnt[v] >= n) return true; // 有负环
            if (!vis[v]) {
                q.push(v);
                vis[v] = 1;
            }
        }
    }
}
return false;
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d", &n, &m);
        for (int i = 1; i <= n; i++) g[i].clear();
        for (int i = 1; i <= m; i++) {
            int u, v, w;
            scanf("%d%d%d", &u, &v, &w);
            g[u].push_back({v, w});
            if (w >= 0) g[v].push_back({u, w});
        }
        puts(spfa() ? "YES" : "NO");
    }
}

```

D18 Tarjan eDCC缩点

```

#include<bits/stdc++.h>
using namespace std;

const int N=500010,M=4000010;
int n,m,a,b;
int h[N],to[M],ne[M],idx=1;
void add(int a,int b){
    to[++idx]=b,ne[idx]=h[a],h[a]=idx;
}
int dfn[N],low[N],tim,stk[N],top,dcc[N],siz[N],cnt,bri[M];
vector<int> d[N];

void tarjan(int x,int ine){
    dfn[x]=low[x]=++tim;stk[++top]=x;

```

```

for(int i=h[x];i;i=ne[i]){
    int y=to[i];
    if(!dfn[y]){
        tarjan(y,i);
        low[x]=min(low[x],low[y]);
        if(low[y]<dfn[x]) bri[i]=bri[i^1]=1;
    }
    else if(i!=(ine^1)) low[x]=min(low[x],dfn[y]);
}

if(dfn[x]==low[x]){
    ++cnt;
    while(1){
        int y=stk[top--];
        // dcc[y]=cnt;
        siz[cnt]++;
        d[cnt].push_back(y);
        if(x==y) break;
    }
}
}

int main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>m;
    while(m--){
        cin>>a>>b;add(a,b);add(b,a);
    }
    for(int i=1;i<=n;i++)if(!dfn[i])tarjan(i,0);
    cout<<cnt<<"\n";
    for(int i=1;i<=cnt;i++){
        cout<<siz[i]<<" ";
        for(int j:d[i]) cout<<j<<" ";
        cout<<"\n";
    }
}

```

D19 Tarjan vDCC缩点

```

#include<bits/stdc++.h>
using namespace std;

const int N=500010;
int n,m,a,b;
vector<int> e[N],ne[N],dcc[N];
int dfn[N],low[N],tot,stk[N],top,cut[N],root,cnt;

void tarjan(int x){
    if(x==root&&!e[x].size()){ //孤立点
        dcc[++cnt].push_back(x);
        return;
    }
    dfn[x]=low[x]=++tot; stk[++top]=x;
    int son=0;
    for(int y:e[x]){
        if(!dfn[y]){ //若y未访问
            tarjan(y);
            low[x]=min(low[x],low[y]);

```

```

        if(low[y]>=dfn[x]){
            son++;
            if(x!=root||son>1)cut[x]=1; //割点

            ++cnt;
            while(1){
                int z=stk[top--];
                dcc[cnt].push_back(z);
                if(z==y) break; //让x留在栈中
            }
            dcc[cnt].push_back(x); //vDCC
        }
    }
    else //若y已访问
        low[x]=min(low[x],dfn[y]);
}
}

int main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>m;
    while(m--){
        cin>>a>>b;
        if(a==b) continue; //忽略自环
        e[a].push_back(b),
        e[b].push_back(a);
    }
    for(root=1;root<=n;root++)if(!dfn[root])tarjan(root);
    cout<<cnt<<"\n";
    for(int i=1;i<=cnt;i++){
        cout<<dcc[i].size()<<" ";
        for(int j:dcc[i])cout<<j<<" ";
        cout<<"\n";
    }
}

```

D21 Dinic最大流

```

// Luogu P3376 【模板】网络最大流
#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>
#define LL long long
#define N 10010
#define M 200010
using namespace std;

int n,m,S,T;
struct edge{LL v,c,ne;}e[M];
int h[N],idx=1; //从2,3开始配对
int d[N],cur[N];

void add(int a,int b,int c){
    e[++idx]={b,c,h[a]};
    h[a]=idx;
}

bool bfs(){ //对点分层，找增广路

```

```

memset(d,0,sizeof d);
queue<int>q;
q.push(S); d[S]=1;
while(q.size()){
    int u=q.front(); q.pop();
    for(int i=h[u];i;i=e[i].ne){
        int v=e[i].v;
        if(d[v]==0 && e[i].c){
            d[v]=d[u]+1;
            q.push(v);
            if(v==T)return true;
        }
    }
}
return false;
}
LL dfs(int u, LL mf){ //多路增广
    if(u==T) return mf;
    LL sum=0;
    for(int i=cur[u];i;i=e[i].ne){
        cur[u]=i; //当前弧优化
        int v=e[i].v;
        if(d[v]==d[u]+1 && e[i].c){
            LL f=dfs(v,min(mf,e[i].c));
            e[i].c-=f;
            e[i^1].c+=f; //更新残留网
            sum+=f; //累加u的流出流量
            mf-=f; //减少u的剩余流量
            if(mf==0)break; //余量优化
        }
    }
    if(sum==0) d[u]=0; //残枝优化
    return sum;
}
LL dinic(){ //累加可行流
    LL flow=0;
    while(bfs()){
        memcpy(cur, h, sizeof h);
        flow+=dfs(S,1e9);
    }
    return flow;
}
int main(){
    int a,b,c;
    scanf("%d%d%d",&n,&m,&S,&T);
    while(m--){
        scanf("%d%d%d",&a,&b,&c);
        add(a,b,c); add(b,a,0);
    }
    printf("%lld\n",dinic());
    return 0;
}

```

D23 最小费用最大流EK

```
// Luogu P3381 【模板】最小费用最大流
#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>
using namespace std;

const int N=5010,M=100010,INF=1e8;
int n,m,S,T;
struct edge{int v,c,w,ne;}e[M];
int h[N],idx=1;//从2,3开始配对
int d[N],mf[N],pre[N],vis[N];
int flow,cost;

void add(int a,int b,int c,int d){
    e[++idx]={b,c,d,h[a]};
    h[a]=idx;
}

bool spfa(){
    memset(d,0x3f,sizeof d);
    memset(mf,0,sizeof mf);
    queue<int> q; q.push(S);
    d[S]=0, mf[S]=INF, vis[S]=1;
    while(q.size()){
        int u=q.front(); q.pop();
        vis[u]=0;
        for(int i=h[u];i;i=e[i].ne){
            int v=e[i].v,c=e[i].c,w=e[i].w;
            if(d[v]>d[u]+w && c){
                d[v]=d[u]+w; //最短路
                pre[v]=i;
                mf[v]=min(mf[u],c);
                if(!vis[v]){
                    q.push(v); vis[v]=1;
                }
            }
        }
    }
    return mf[T]>0;
}

void EK(){
    while(spfa()){
        for(int v=T;v!=S;){
            int i=pre[v];
            e[i].c-=mf[T];
            e[i^1].c+=mf[T];
            v=e[i^1].v;
        }
        flow+=mf[T]; //累加可行流
        cost+=mf[T]*d[T]; //累加费用
    }
}

int main(){
    scanf("%d%d%d%d",&n,&m,&S,&T);
    int a,b,c,d;
```

```

while(m --){
    scanf("%d%d%d%d",&a,&b,&c,&d);
    add(a,b,c,d);
    add(b,a,0,-d);
}
EK();
printf("%d %d\n",flow,cost);
return 0;
}

```

D27 二分图最大匹配

```

//
#include <iostream>
#include <cstring>
#include <algorithm>
#include <queue>
#define N 1010
#define M 2000010
using namespace std;

int n,m,k,S,T;
struct edge{int v,c,ne;}e[M];
int h[N],idx=1; //从2,3开始配对
int d[N],cur[N];

void add(int a,int b,int c){
    e[++idx]={b,c,h[a]};
    h[a]=idx;
}

bool bfs(){ //对点分层，找增广路
    memset(d,0,sizeof d);
    queue<int>q;
    q.push(S); d[S]=1;
    while(q.size()){
        int u=q.front(); q.pop();
        for(int i=h[u];i;i=e[i].ne){
            int v=e[i].v;
            if(d[v]==0 && e[i].c){
                d[v]=d[u]+1;
                q.push(v);
                if(v==T)return true;
            }
        }
    }
    return false;
}

int dfs(int u, int mf){ //多路增广
    if(u==T) return mf;
    int sum=0;
    for(int i=cur[u];i;i=e[i].ne){
        cur[u]=i; //当前弧优化
        int v=e[i].v;
        if(d[v]==d[u]+1 && e[i].c){
            int f=dfs(v,min(mf,e[i].c));
            e[i].c-=f;
            e[i^1].c+=f; //更新残留网
        }
    }
    return sum;
}

```

```

        sum+=f; //累加u的流出流量
        mf-=f; //减少u的剩余流量
        if(mf==0)break; //余量优化
    }
}
if(sum==0) d[u]=0; //残枝优化
return sum;
}
int dinic(){ //累加可行流
    int flow=0;
    while(bfs()){
        memcpy(cur, h, sizeof h);
        flow+=dfs(s,1e9);
    }
    return flow;
}
int main(){
    int a,b,c;
    scanf("%d%d%d",&n,&m,&k);
    while(k--){
        scanf("%d%d",&a,&b);
        add(a,b+n,1);add(b+n,a,0);
    }
    S=0;T=n+m+1;
    for(int i=1;i<=n;i++)
        add(S,i,1),add(i,S,0);
    for(int i=1;i<=m;i++)
        add(i+n,T,1),add(T,i+n,0);
    printf("%lld\n",dinic());
    return 0;
}

```

D99 Kruskal重构树

```

/*EHnotgod-----
.....#####.....#.....#.....
.....#.....#.....#.....
.....#####.....#####
.....#.....#.....#.....
.....#####.....#.....#.....
*/
#include <bits/stdc++.h>
using namespace std;
#define endl "\n"
#define int long long
#define range(i, a, b) for (int i = (a); i < (b); ++i)
#define def(name, ...) auto name = [&](__VA_ARGS__)

const int N=200006;
int n, m, q;
struct edge{
    int u,v,w;
    bool operator<(const edge &t)const
    {return w < t.w;}
}edge[N];
int fa2[N],ans,cnt;
vector<int> value(N, 0);

```

```

vector<vector<int>> e(N, vector<int>());
int find(int x){
    if(fa2[x]==x) return x;
    return fa2[x]=find(fa2[x]);
}
void kruskal(){
    sort(edge,edge+m);
    cnt = n;
    for(int i=1;i<N;i++)fa2[i]=i;
    for(int i=0; i<m; i++){
        int x = find(edge[i].u);
        int y = find(edge[i].v);
        if(x!=y){
            cnt++;
            fa2[x]=cnt;
            fa2[y]=cnt;
            value[cnt] = edge[i].w;

            e[x].push_back(cnt);
            e[y].push_back(cnt);
            e[cnt].push_back(x);
            e[cnt].push_back(y);
        }
    }
}
int f[N][22],dep[N];

void dfs(int u,int fa){
    f[u][0]=fa; dep[u]=dep[fa]+1;
    for(int i=1;i<=20;i++) //u的2,4,8...祖先
        f[u][i]=f[f[u][i-1]][i-1];
    for(int v:e[u])
        if(v!=fa) dfs(v,u);
}
int lca(int u,int v){
    if(dep[u]<dep[v]) swap(u,v);
    for(int i=20;~i;i--) //u先大步后小步向上跳，直到与v同层
        if(dep[f[u][i]]>=dep[v]) u=f[u][i];
    if(u==v) return v;
    for(int i=20;~i;i--) //u,v一起向上跳，直到lca的下面
        if(f[u][i]!=f[v][i]) u=f[u][i],v=f[v][i];
    return f[u][0];
}
// vector<int> a(n, 0);
// vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
void solve() {
    cin >> n >> m >> q;
    for (int i = 0; i < m; i++){
        cin >> edge[i].u >> edge[i].v >> edge[i].w;
    }
    kruskal();

    dfs(cnt, 0);

    while (q--){
        int u, v; cin >> u >> v;
        int uv = lca(u, v);
        cout << value[uv] << endl;
    }
}

```

```

    }
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int t;
    t = 1;
    while (t--) {
        solve();
    }
    return 0;
}

```

E 动态规划

E43 单调队列优化dp

```

// 单调队列+DP O(n)
#include<bits/stdc++.h>
using namespace std;

const int N=200010;
int n,m,a[N];
int q[N],f[N];

int main(){
    cin>>n>>m;
    for(int i=1; i<=n; i++) cin>>a[i];

    int ans=2e9;
    for(int i=1,h=1,t=0; i<=n; i++){
        while(h<=t && q[h]<i-m) h++;
        while(h<=t && f[q[t]]>=f[i-1]) t--;
        q[++t]=i-1;
        f[i]=f[q[h]]+a[i];
        if(i>=n-m+1) ans=min(ans,f[i]);
    }
    cout<<ans;
}

```

E51 斜率优化DP

```

#include <iostream>
#include <cstring>
#include <algorithm>
using namespace std;

typedef long long LL;
const int N = 500010;
int n,m,q[N];
LL s[N],f[N];

LL dy(int i,int j){return f[i]+s[i]*s[i]-f[j]-s[j]*s[j];}

```

```

LL dx(int i,int j){return s[i]-s[j];}
int main(){
    while(~scanf("%d%d",&n,&m)){
        for(int i=1;i<=n;i++)scanf("%lld",&s[i]),s[i]+=s[i-1];

        int h=1,t=0;
        for(int i=1;i<=n;i++){
            while(h<t && dy(i-1,q[t])*dx(q[t],q[t-1])
                <=dx(i-1,q[t])*dy(q[t],q[t-1])) t--;
            q[++t]=i-1;
            while(h<t && dy(q[h+1],q[h])
                <=dx(q[h+1],q[h])*2*s[i]) h++;
            int j=q[h];
            f[i]=f[j]+(s[i]-s[j])*(s[i]-s[j])+m;
        }
        printf("%lld\n",f[n]);
    }
}

```

F 字符串

F5 马拉车算法-最长回文子串

```

#include <iostream>
#include <cstring>
#include <algorithm>
using namespace std;

const int N=3e7;
char a[N],s[N];
int d[N]; //回文半径函数

void get_d(char*s,int n){
    d[1]=1;
    for(int i=2,l,r=1;i<=n;i++){
        if(i<=r)d[i]=min(d[r-i+1],r-i+1);
        while(s[i-d[i]]==s[i+d[i]])d[i]++;
        if(i+d[i]-1>r)l=i-d[i]+1,r=i+d[i]-1;
        // printf("i=%d d=%d [%d %d]\n",i,d[i],l,r);
    }
}

int main(){
    //改造串
    scanf("%s",a+1);
    int n=strlen(a+1),k=0;
    s[0]='$',s[++k]='#';
    for(int i=1;i<=n;i++)
        s[++k]=a[i],s[++k]='#';
    n=k;

    get_d(s,n); //计算d函数
    int ans=0;
    for(int i=1;i<=n;i++)
        ans=max(ans,d[i]);
    printf("%d\n",ans-1);
    return 0;
}

```

```
}
```

G 数学

G12 莫比乌斯函数

```
#include <iostream>
using namespace std;

const int N = 1000010;
int p[N], vis[N], cnt;
int mu[N];

void get_mu(int n){//筛法求莫比乌斯函数
    mu[1] = 1;
    for(int i=2; i<=n; i++){
        if(!vis[i]){
            p[++cnt] = i;
            mu[i] = -1;
        }
        for(int j=1; i*p[j]<=n; j++){
            int m = i*p[j];
            vis[m] = 1;
            if(i%p[j] == 0){
                mu[m] = 0;
                break;
            }
            else
                mu[m] = -mu[i];
        }
    }
}

int main(){
    int n;
    cin >> n;
    get_mu(n);
    for(int i=1; i<=n; i++)
        printf("%d\n",mu[i]);
    return 0;
}
```

G24 高斯约旦消元法

```
#include<iostream>
#include<cstdio>
#include<cmath>
#define LL long long
using namespace std;

const int N=405,P=1e9+7;
int n;
LL a[N][N<<1];

LL quickpow(LL a, LL b){
    LL ans = 1;
```

```

while(b){
    if(b & 1) ans = ans*a%P;
    a = a*a%P;
    b >>= 1;
}
return ans;
}
bool Gauss_Jordan(){
    for(int i=1;i<=n;++i){ //枚举主元的行列
        int r = i;
        for(int k=i; k<=n; ++k) //找非0行
            if(a[k][i]) {r=k; break;}
        if(r!=i) swap(a[r],a[i]); //换行
        if(!a[i][i]) return 0;

        int x=quickpow(a[i][i],P-2); //求逆元
        for(int k=1; k<=n; ++k){ //对角化
            if(k == i) continue;
            int t=a[k][i]*x%P;
            for(int j=i; j<=2*n; ++j)
                a[k][j]=(a[k][j]-t*a[i][j])%P+P)%P;
        }
        for(int j=1; j<=2*n; ++j) //除以主元
            a[i][j]=(a[i][j]*x%P);
    }
    return 1;
}
int main(){
    scanf("%d",&n);
    for(int i=1; i<=n; ++i)
        for(int j=1; j<=n; ++j)
            scanf("%lld",&a[i][j]),a[i][i+n]=1;
    if(Gauss_Jordan())
        for(int i=1; i<=n; ++i){
            for(int j=n+1; j<=2*n; ++j)
                printf("%lld ",a[i][j]);
            puts("");
        }
    else puts("No Solution");
    return 0;
}

```

G26 求组合数-线性逆推

```

# 逆推法（阶乘 + 快速幂 + 模逆）
MOD = 10**9 + 7
N = 10**6 # 最大 n

# 预处理阶乘和逆阶乘
fac = [1] * (N + 1)
inv = [1] * (N + 1)

# 计算阶乘
for i in range(1, N + 1):
    fac[i] = fac[i - 1] * i % MOD

# 快速幂

```

```

def qpow(a, b):
    res = 1
    while b:
        if b & 1:
            res = res * a % MOD
        a = a * a % MOD
        b >>= 1
    return res

# 计算逆阶乘
inv[N] = qpow(fac[N], MOD - 2)
for i in range(N, 0, -1):
    inv[i - 1] = inv[i] * i % MOD

# 组合数函数
def C(n, k):
    if k < 0 or k > n:
        return 0
    return fac[n] * inv[k] % MOD * inv[n - k] % MOD

# 使用:
print(C(10, 3)) # 输出 120

```

G51 三角剖分

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <cmath>
#define x first
#define y second
using namespace std;

typedef pair<double,double> Point;
const double eps=1e-8;
const double PI=acos(-1.0);
double R;
Point p[4],o; //顶点和圆心

Point operator+(Point a,Point b){ //向量+
    return Point(a.x+b.x,a.y+b.y);
}
Point operator-(Point a,Point b){ //向量-
    return Point(a.x-b.x,a.y-b.y);
}
Point operator*(Point a,double t){ //数乘
    return Point(a.x*t,a.y*t);
}
Point operator/(Point a,double t){ //数除
    return Point(a.x/t,a.y/t);
}
double operator*(Point a,Point b){ //叉积
    return a.x*b.y-a.y*b.x;
}

```

```

double operator&(Point a,Point b){ //点积
    return a.x*b.x+a.y*b.y;
}
double len(Point a){ //模长
    return sqrt(a&a);
}
double dis(Point a,Point b){ //距离
    return len(b-a);
}
Point getNode(Point a,Point u,Point b,Point v){ //直线交点
    double t=(a-b)*v/(v*u);
    return a+u*t;
}
Point rotate(Point a,double b){ //逆转角
    return Point(a.x*cos(b)-a.y*sin(b),a.x*sin(b)+a.y*cos(b));
}
bool onSegment(Point p,Point a,Point b){ //p在线段ab上
    return fabs((a-p)*(b-p))<eps && ((a-p)&(b-p))<=0;
}
Point norm(Point a){ //单位向量
    return a/len(a);
}
double getDP2(Point a,Point b,Point& pa,Point& pb){
    Point e=getNode(a,b-a,o,rotate(b-a,PI/2)); //垂足
    double d=dis(o,e);
    if(!onSegment(e,a,b)) d=min(dis(o,a),dis(o,b));
    if(R<=d) return d; //线段在圆外
    double len=sqrt(R*R-dis(o,e)*dis(o,e));
    pa=e+norm(a-b)*len;
    pb=e+norm(b-a)*len; //pa,pb:线段与圆的两交点
    return d; //d:圆心到线段的最近距离
}
double sector(Point a,Point b){ //扇形面积
    double angle=acos((a&b)/len(a)/len(b)); //[0,Pi]
    if(a*b<0) angle=-angle;
    return R*R*angle/2;
}
double getArea(Point a,Point b){ //面积的计算
    if(fabs(a*b)<eps) return 0; //共线
    double da=dis(o,a),db=dis(o,b);
    if(R>=da && R>=db) return a*b/2; //ab在圆内
    Point pa,pb;
    double d=getDP2(a,b,pa,pb); //d:圆心到线段的最近距离
    if(R<=d) return sector(a,b); //ab在圆外
    if(R>=da) return a*pb/2+sector(pb,b); //a在圆内
    if(R>=db) return sector(a,pa)+pa*b/2; //b在圆内
    return sector(a,pa)+pa*pb/2+sector(pb,b); //ab是割线
}
int main(){
    while(scanf("%lf%lf%lf%lf%lf%lf%lf%lf%lf",
        &p[0].x,&p[0].y,&p[1].x,&p[1].y,&p[2].x,&p[2].y,&o.x,&o.y,&R)!=-1){
        for(int i=0;i<3;i++) p[i].x-=o.x,p[i].y-=o.y; //三角形顶点平移
        o=Point(0,0); //圆心平移到原点
        double res=0;
        for(int i=0;i<3;i++) res+=getArea(p[i],p[(i+1)%3]);
        printf("%.2lf\n",fabs(res)); //点可能顺时针
    }
    return 0;
}

```

```
}
```

G57 自适应辛普森积分

```
#include <iostream>
#include <cstring>
#include <algorithm>
#include <cmath>
using namespace std;

const double eps=1e-10;
double a,b,c,d,l,r;

double f(double x){ //积分函数
    return (c*x+d)/(a*x+b);
}
double simpson(double l,double r){//辛普森公式
    return (r-l)*(f(l)+f(r)+4*f((l+r)/2))/6;
}
double asr(double l,double r,double ans){//自适应
    auto m=(l+r)/2,a=simpson(l,m),b=simpson(m,r);
    if(fabs(a+b-ans)<eps) return ans;
    return asr(l,m,a)+asr(m,r,b);
}
int main(){
    scanf("%lf%lf%lf%lf%lf%lf",&a,&b,&c,&d,&l,&r);
    printf("%.6lf",asr(l,r,simpson(l,r)));
    return 0;
}
```

G60 有向图游戏-SG函数

```
#include <cstdio>
#include <cstring>
#include <set>
using namespace std;

const int N=2005,M=10005;
int n,m,k,a,b,x;
int h[N],to[M],ne[M],tot; //邻接表
int f[N];

void add(int a,int b){
    to[++tot]=b,ne[tot]=h[a],h[a]=tot;
}
int sg(int u){
    // 记忆化搜索
    if(f[u]!=-1) return f[u];
    // 把子节点的sg值插入集合
    set<int> S;
    for(int i=h[u];i;i=ne[i])
        S.insert(sg(to[i]));
    // mex运算求当前节点的sg值并记忆
    for(int i=0; ;i++)
        if(!S.count(i)) return f[u]=i;
}
```

```

int main(){
    scanf("%d%d%d",&n,&m,&k);
    for(int i=0;i<m;i++)
        scanf("%d%d",&a,&b), add(a,b);
    memset(f,-1,sizeof f);
    int res=0;
    for(int i=0;i<k;i++)
        scanf("%d",&x), res^=sg(x);
    if(res) puts("win");
    else puts("lose");
}

```

G74 拉格朗日插值法

```

// 拉格朗日插值法 O(n^2)
#include <iostream>
#include <cstring>
#include <algorithm>
using namespace std;

#define LL long long
const LL mod=998244353;
LL n,k,ans;
LL x[2005],y[2005];

LL ksm(LL a,LL b){
    LL s=1;
    while(b){
        if(b&1)s=s*a%mod;
        a=a*a%mod;
        b>>=1;
    }
    return s;
}

int main(){
    cin>>n>>k;
    for(int i=1;i<=n;i++)cin>>x[i]>>y[i];
    for(int i=1;i<=n;i++){
        LL a=y[i],b=1;
        for(int j=1;j<=n;j++){
            if(i==j) continue;
            a=a*(k-x[j])%mod;
            b=b*(x[i]-x[j])%mod;
        }
        ans=(ans+a*ksm(b,mod-2)%mod)%mod;
    }
    cout<<(ans+mod)%mod;
}

```